

OOP (Object-Oriented Programming)

Conceitos

Professor Antonio Marcos Alvarez
Técnicas de Programação

Programação Estruturada e Programação Orientado a Objetos

Programação Estruturada

- ▶ **Base:**
 - **Sequência:** Uma tarefa é executada após a outra, linearmente.
 - **Decisão:** A partir de um teste lógico, determinado trecho de código é executado, ou não.
 - **Iteração:** A partir de um teste lógico, determinado trecho de código é repetido por um número finito de vezes.
- ▶ **Vantagens**
 - É fácil de entender. Ainda muito usada em cursos introdutórios de programação.
 - Execução mais rápida.
- ▶ **Desvantagens**
 - Baixa reutilização de código

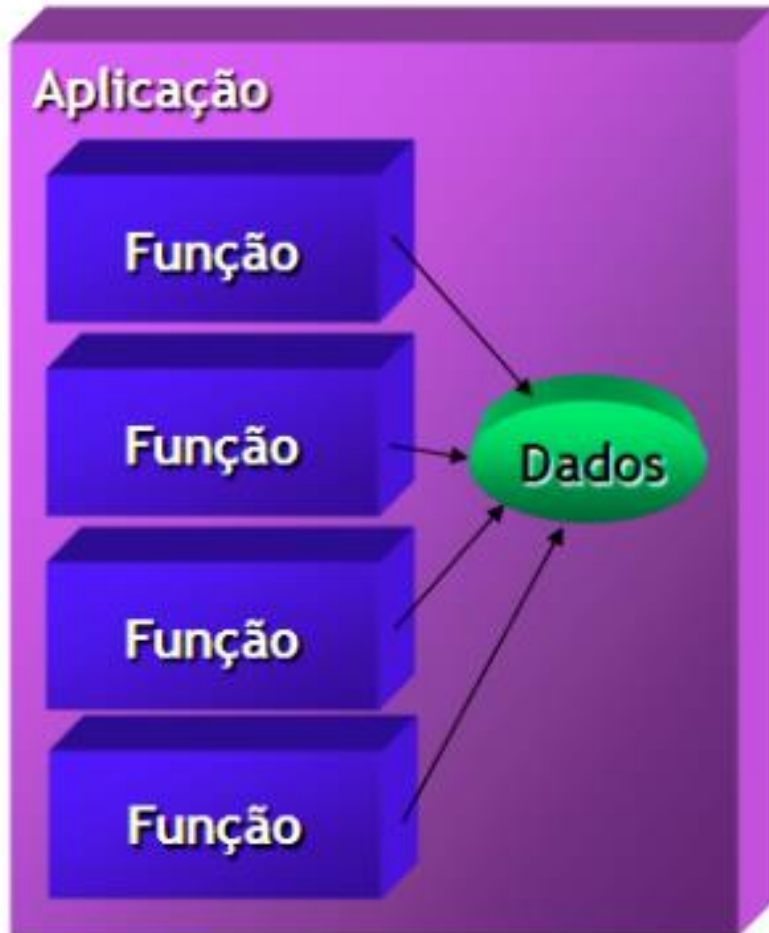
Programação Orientada a Objetos

- ▶ **Base**
 - Classes e Objetos
 - Métodos e Atributos
- ▶ **Vantagens**
 - Melhor organização do código
 - Bom reaproveitamento de código
- ▶ **Desvantagens**
 - Desempenho mais baixo que o paradigma estruturado

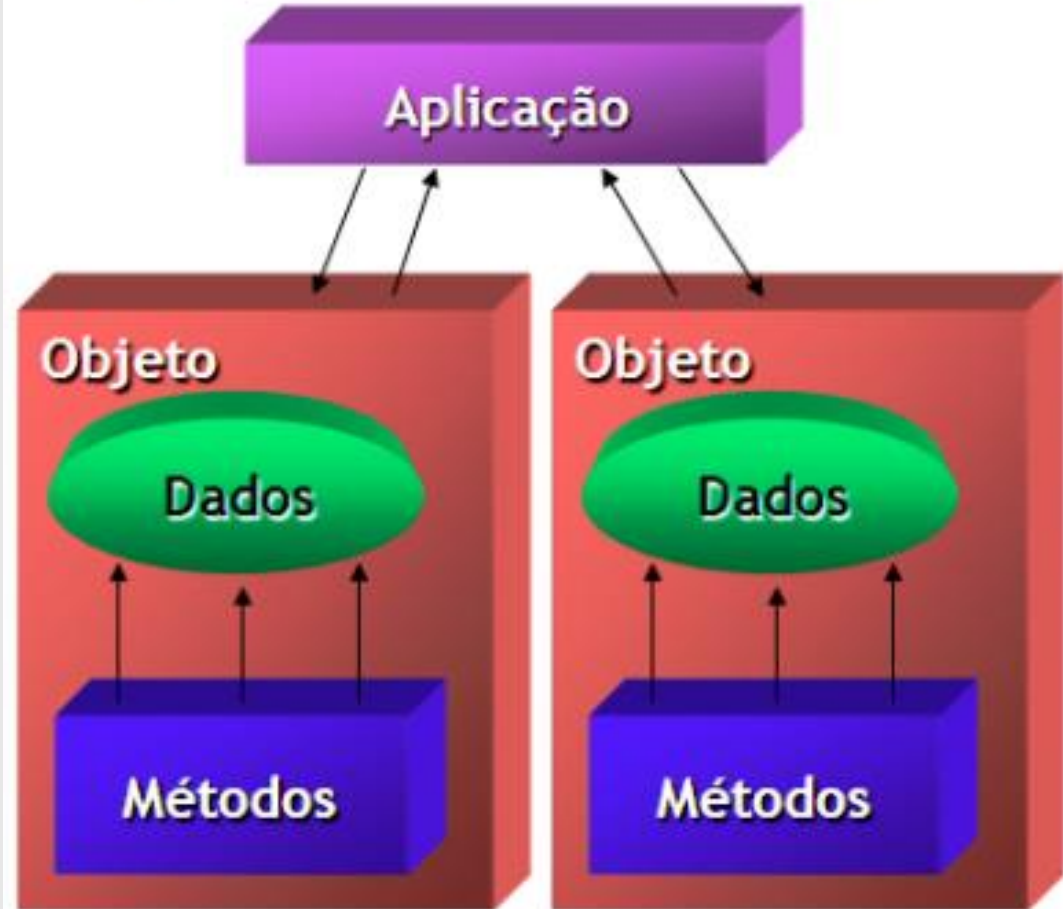
- Programação Estruturada (PE, ou SP) é um padrão ou paradigma de programação da engenharia de softwares, com ênfase em sequência, decisão e, iteração (sub-rotinas, laços de repetição, condicionais e, estruturas em bloco).
- Programação orientada a objetos (POO, ou OOP) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados na forma de campos, também conhecidos como atributos, e códigos, na forma de procedimentos, também conhecidos como métodos. Uma característica de objetos é que um procedimento de objeto pode acessar, e geralmente modificar, os campos de dados do objeto com o qual eles estão associados (objetos possuem uma noção de "this" (este) ou "self" (próprio)).

Programação Estruturada e Programação Orientado a Objetos

Estruturada



Orientação a Objetos



Origem OOP

A Programação Orientada ao Objeto (Object-Oriented Programming) foi concebida há muito tempo atrás (no início da década de 70), a sua origem vem da linguagem **Simula** (***Simula Language***), concebida na Noruega na década de 60, e como o nome indica, foi criada para fazer simulações; entretanto, seu uso alavancou um conceito que até então passava despercebido pela maioria dos projetistas: a similaridade com o mundo real.

A primeira linguagem de programação a implementar os conceitos de OOP foi a linguagem **SIMULA-68**; em seguida surgiu a linguagem **Smalltalk**; criada pela Xerox, que pode ser considerada a linguagem que popularizou e incentivou o emprego da OOP.

O resultado foi uma linguagem de pura linhagem OO, poderosíssima, que implementa todos os conceitos de OO, o que não acontece com as chamadas linguagens OO híbridas que implementam apenas alguns conceitos de orientação ao objeto.

Fundamentalmente o que se deseja com esta metodologia são basicamente duas características: reutilização de código e modularidade de escrita. Formalmente, para ser considerada uma linguagem OO, esta precisa implementar quatro conceitos básicos: abstração, encapsulamento, herança e polimorfismo.

Outras linguagens (Históricas)

C++: Foi inicialmente desenvolvido por Bjarne Stroustrup dos Bell Labs, durante a década de 1980 com o objetivo implementar uma versão distribuída do núcleo Unix. Algumas questões no projeto de C++: Ser melhor do que C; Suportar abstração de dados; Suportar programação orientada a objetos; C++ foi projetada para dar suporte a abstração de dados e programação orientada a objetos; C++ não impõe um paradigma.

Ada: Uma linguagem de programação criada através de um concurso realizado pelo U.S. Department of Defense (DoD). O principal projetista da equipe foi o francês Jean Ichbiah. Esse concurso foi feito para por ordem na situação, o “DoD” em 1974 usava cerca de 450 linguagens ou dialetos de programação. A linguagem foi primeiramente padronizada em 1983 pelo ANSI e em 1985 a Organização Internacional de Padronização (ISO).

Eiffel: Uma Linguagem de Programação avançada, puramente orientada a objeto que enfatiza o projeto e construção de software reutilizável e de alta qualidade. Eiffel foi criada por Bertrand Meyer que tinha uma extensa experiência com programação orientada a objeto, particularmente com SIMULA.

Object Pascal: Uma linguagem orientada a objetos, isto é, todas as informações são tratadas como objetos e todos estes objetos pertencem a uma classe, que são categorias de objetos. Delphi / Kylix / Lazarus são exemplos de ferramentas que utilizam esta linguagem.

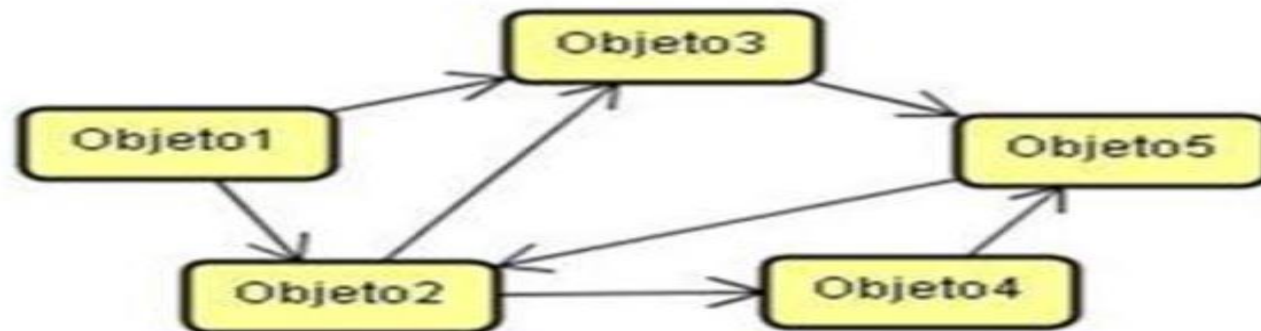
Delphi: IDE, compilador, uma linguagem de programação (Base posterior ao Pascal), produzido anteriormente pela Borland Software Corporation e atualmente pela “Embarcadero”.

Java: O Java é ao mesmo tempo um ambiente e uma linguagem de programação desenvolvida pela Sun Microsystems Inc. Trata-se de mais um representante da geração de linguagens orientadas a objetos e foi projetado para resolver os problemas da área de programação cliente/servidor. Os aplicativos em Java são compilados em um código de bytes (“bytecodes”) independente de arquitetura. Esse código de bytes pode então ser executado em qualquer plataforma que suporte um interpretador Java. O Java requer somente uma fonte e um binário e, mesmo assim, é capaz de funcionar em diversas plataformas, o que faz dele um sonho de todos os que realizam manutenção em programas.

Plataforma .NET: A plataforma .NET baseia-se em um dos princípios utilizados na tecnologia Java (Just In Time Compiler – JIT), os programas desenvolvidos para ela são duplo-compilados (compilados duas vezes), uma na distribuição (gerando um código que é conhecido como “bytecodes”) e outra na execução. Um programa é escrito em qualquer das mais de vinte linguagens de programação disponíveis para a plataforma, o código fonte gerado pelo programador é então compilado pela linguagem escolhida gerando um código intermediário em uma linguagem chamada MSIL (Microsoft Intermediate Language). Este novo código fonte gera um arquivo na linguagem de baixo nível Assembly, de acordo com o tipo de projeto.

Conceitos OOP

- ▶ “Uma nova maneira de pensar os problemas utilizando conceitos do Mundo Real. O componente fundamental é o objeto que combina estrutura e comportamento em uma única entidade”
 - Raumbaugh
- ▶ “Sistema orientado a objetos é uma coleção de objetos que interagem entre si”
 - Bertrand Meyer



Orientação a Objetos (Abstração)

▶ Abstração:

- Habilidade de se concentrar nos aspectos essenciais do sistema, ou um contexto qualquer, ignorando o que é supérfluo



Conceitos:

Classe

- Representação de um conjunto de objetos com características afins. Definição do comportamento dos objetos (métodos) e seus atributos (atributos).

Objeto

- Uma instância de uma classe.
- Armazenamento de estados através de seus atributos e reação a mensagens enviadas por outros objetos.

Herança

- Mecanismo pela qual uma classe (sub-classe) pode estender outra classe (super-classe), estendendo seus comportamentos e atributos.

Polimorfismo

- Princípio pelo qual as instâncias de duas classes ou mais classes derivadas de uma mesma super-classe podem invocar métodos com a mesma assinatura, mas com comportamentos distintos.

Encapsulamento

- Proibição do acesso direto ao estado de um objeto, disponibilizando apenas métodos que alterem esses estados na interface pública.

Objetos

▶ O que é:

- Representação computacional de algo do mundo real
 - Concreto
 - Abstrato

▶ Abstração

- Transformar aquilo que observamos realidade para a virtualidade

Objetos

- ▶ Concretos

- Cão
- Moto
- Casa

- ▶ Abstratos

- Música
- Transação Bancária

- ▶ Modelo

- Características + Comportamento

- ▶ Estado

- Atributos (Características)

- ▶ Operações

- Métodos (Comportamentos)

- ▶ Identidade

- Dois objetos com estado e operações precisamente idênticos não são iguais

- ▶ Operações podem mudar os valores dos atributos **assim mudando o estado de um objeto.**

Método e Atributos



- ▶ Atributos

- Raça: Poodle
- Nome: Rex
- Peso: 5 quilos

- ▶ Método

- Latir
- Comer
- Dormir

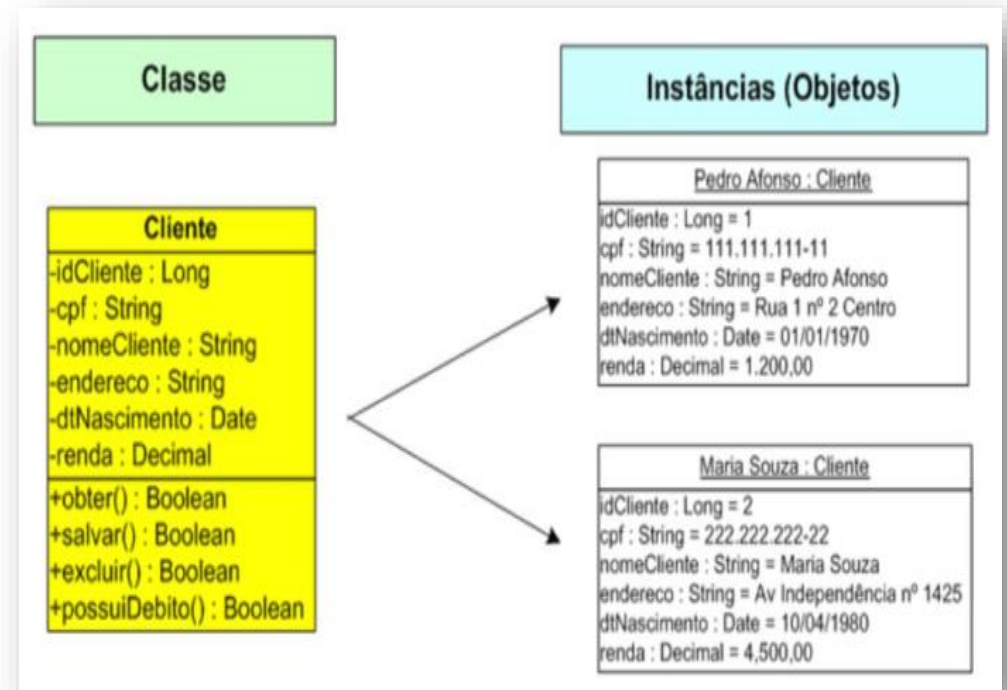


- Potência: 500cc
- Modelo: Honda
- Ano: 1998

- Acelerar
- Frear
- Abastecer

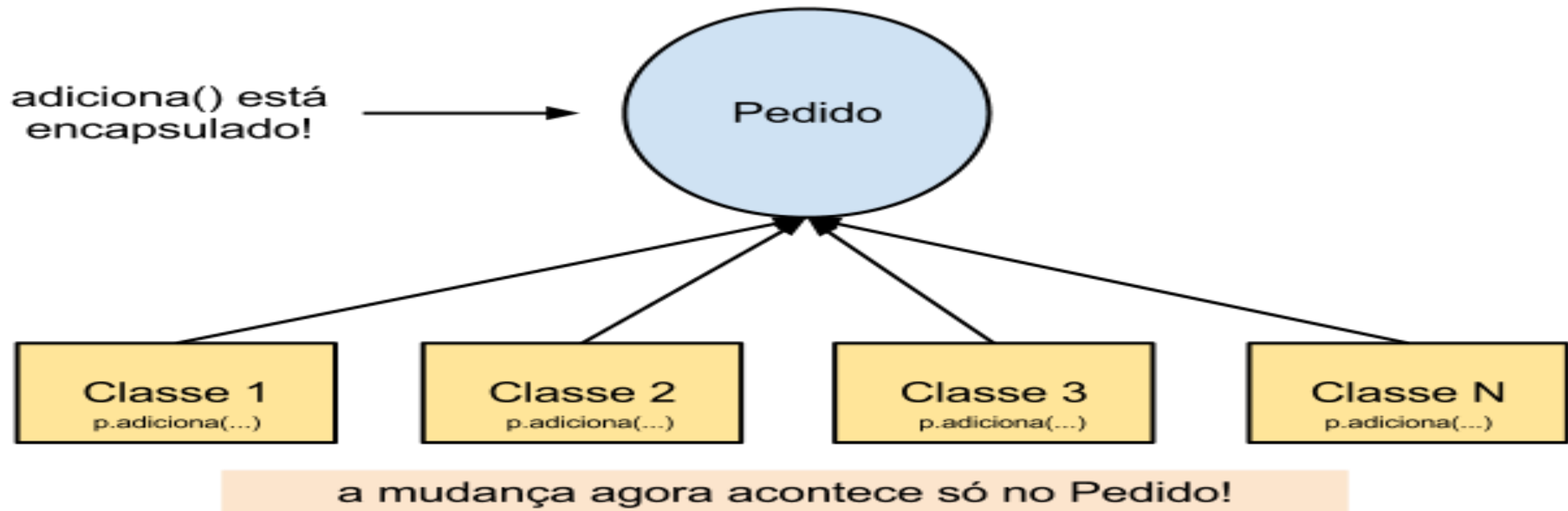
Classe

- ▶ Conjunto de objetos:
 - Características semelhantes
 - Comportamento comum
 - Interação com outros objetos
- ▶ Uma **classe** é a forma para **criação** de objetos
- ▶ Objetos são representações concretas (instâncias) de uma classe



Encapsulamento

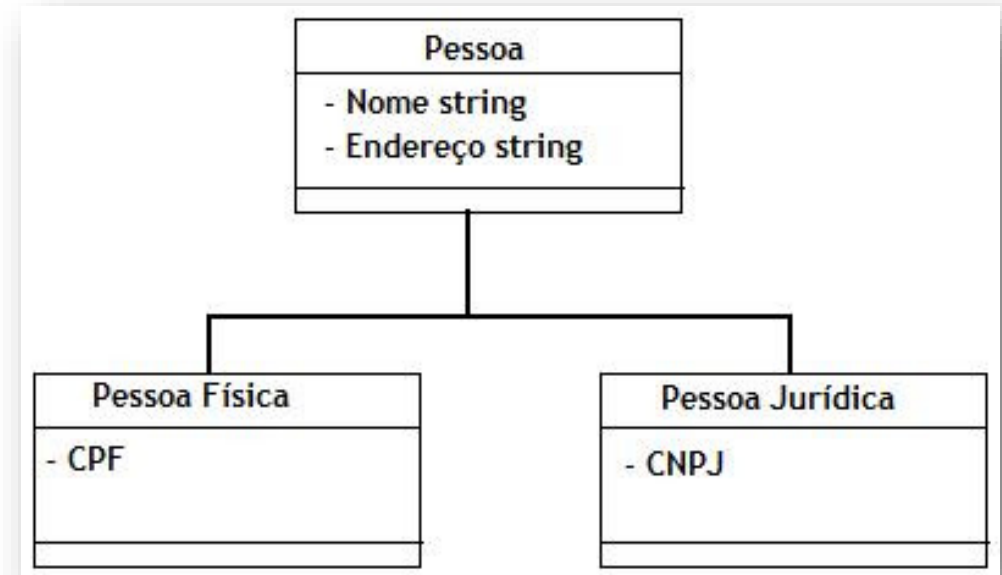
Encapsulamento vem de **encapsular**, que em **programação orientada a objetos** significa separar o programa em partes, o mais isolado possível. A ideia é tornar o software mais flexível, fácil de modificar e de criar novas implementações. O **Encapsulamento** serve para controlar o acesso aos atributos e métodos de uma classe.



Herança

Herança é um princípio de orientação a **objetos**, que permite que classes compartilhem atributos e métodos, através de "**heranças**". Ela é usada na intenção de reaproveitar código ou comportamento generalizado ou especializar operações ou atributos. O conceito de **herança** de várias classes é conhecido como **herança** múltipla.

- A ideia é reutilizar o código de uma determinada classe em outras classes.
- Aplicando herança:
 - Teríamos a classe *Servico* com os atributos e métodos que todos os serviços comuns;
 - Uma classe para cada serviço com os atributos e métodos específicos do determinado serviço.
- Como funciona:
 - As classes específicas seriam "ligadas" de alguma forma à classe *Servico* para reaproveitar o código nela definido.



Polimorfismo

Polimorfismo é o princípio pelo qual duas ou mais classes derivadas da mesma superclasse podem invocar métodos que têm a mesma assinatura, mas comportamentos distintos.

Na programação orientada a objetos, o polimorfismo permite que referências de tipos de classes mais abstratas representem o comportamento das classes concretas que referenciam. Assim, é possível tratar vários tipos de maneira homogênea. O termo polimorfismo é originário do grego e significa "muitas formas".



Como o gerente é um funcionário, podemos nos referir a ele como sendo um Funcionário.

Visibilidades (Tipos de Classes)

– Private

Somente a classe tem acesso

Não é transmitido por herança

~ Default ou Friendly

Acesso a classe inteira

Visível para as classes do mesmo pacote

Só é transmitido por herança em classes do mesmo pacote

Protected

Visível em toda a classe

Visível em todas as classes de um pacote

Transmitido por herança

+ Public

Visível irrestritamente

Na Programação Orientada a Objetos contamos com diversos tipos de classes como: Públicas, Privadas, Protegidas, Estáticas, Abstratas, Seladas, Herdadas, Padrão ou Amigáveis, entre outras... Acima citamos algumas.